Com S // Cpr E // Math 5250

Numerical Analysis of High-Performance Computing

Instructor: Songting Luo

Lecture 20: OpenMP Case Study: Adaptive Quadrature

Outline

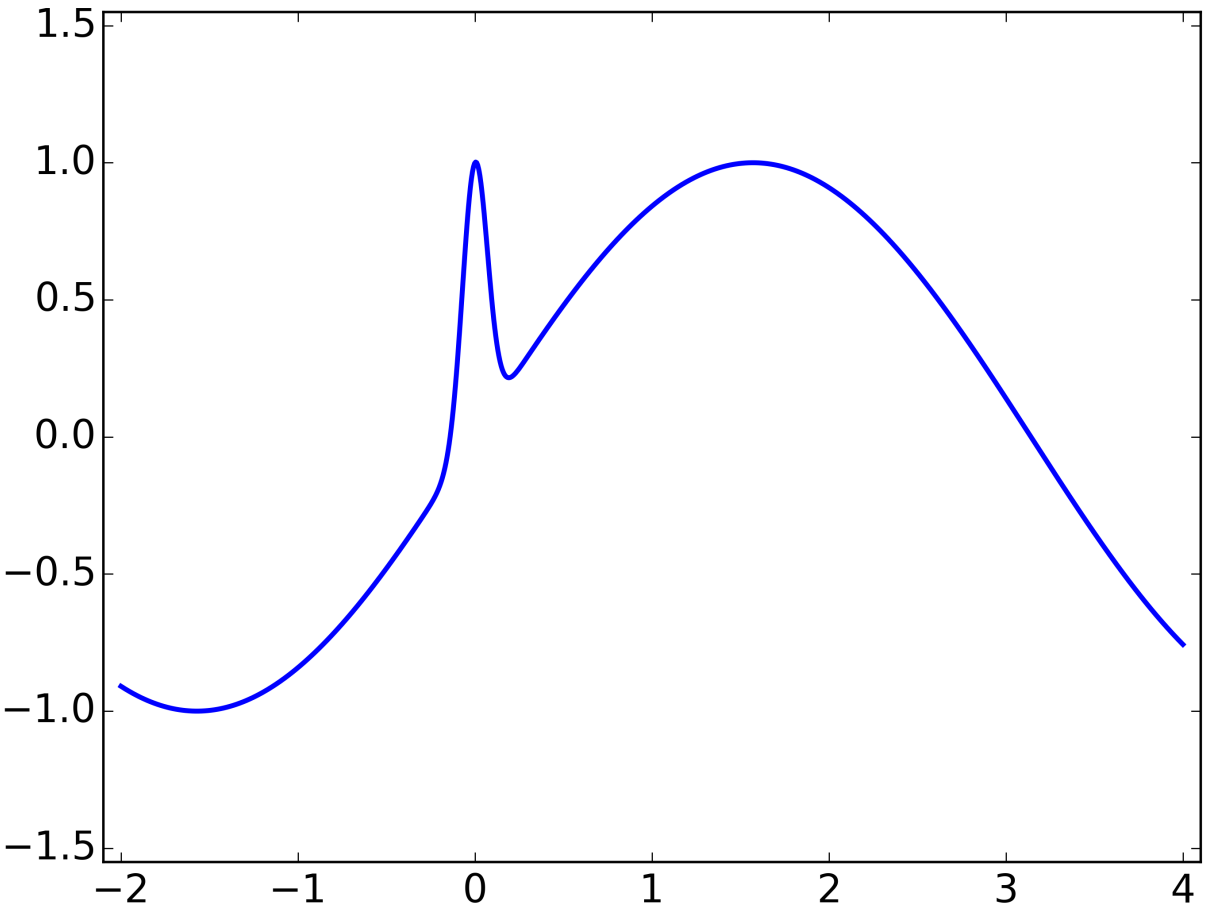
1. OpenMP Case Study: Adaptive Quadrature

OpenMP Case Study: Adaptive Quadrature

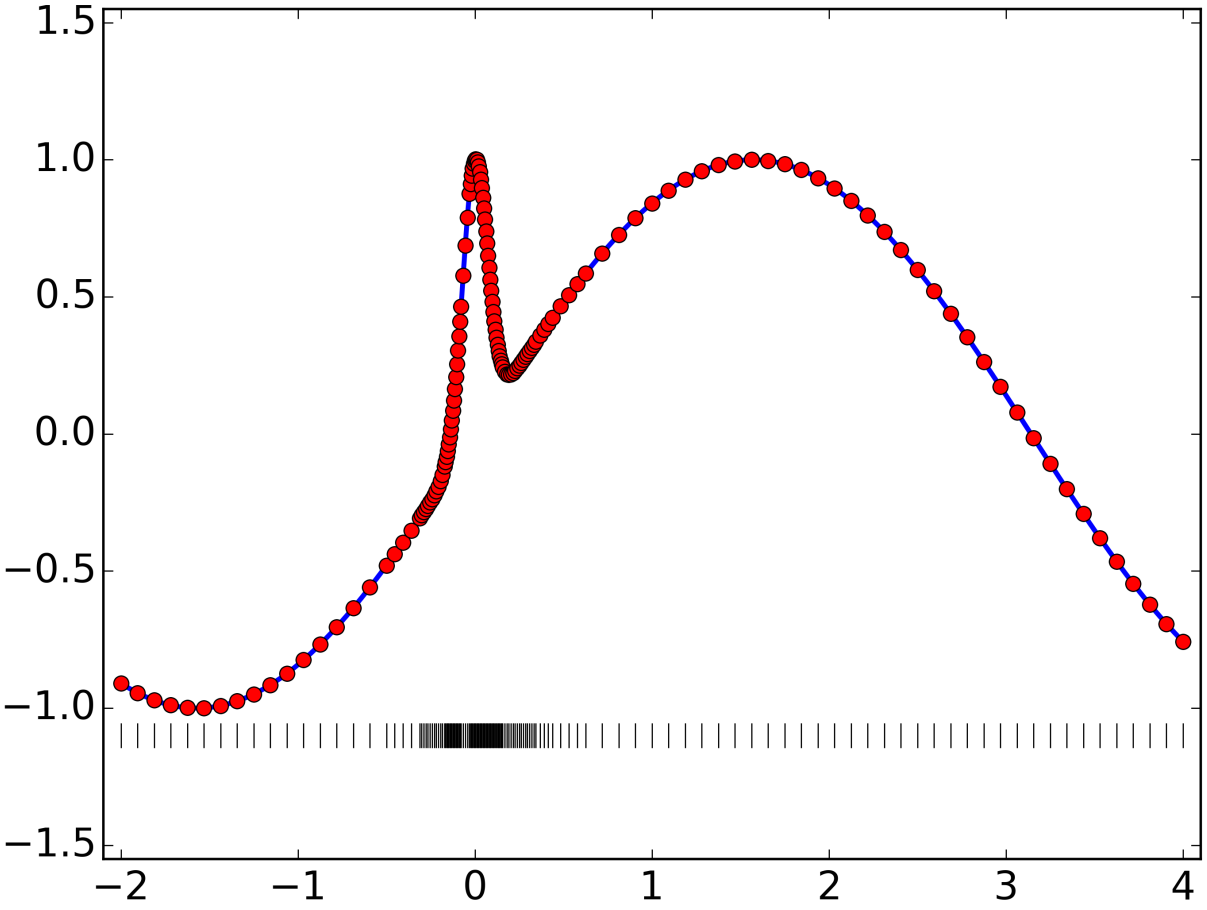
Adaptive quadrature

Problem: approximate

$$\int_{-2}^{4} \left[e^{-100x^2} + \sin(x) \right] dx = \left[\frac{\sqrt{\pi}}{20} \mathrm{erf}(10x) - \cos(x) \right]_{-2}^{4} \approx 0.4147421694070211$$



Adaptive quadrature



Idea: Subdivide into subintervals and apply Simpson's rule on each.

Use larger intervals where $f(x)$ is smoother. **Automate!**

Error estimation

Key: the key for any adaptive scheme is to have an accurate error estimate.

Consider Simpson's rule on the interval $[a, b]$:

$$Q(a,b) := \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right] \approx \int_a^b f(x)\,dx$$

Now consider another approximation based on splitting the integral into equal pieces

$$\int_a^b f(x)\,dx = \int_a^c f(x)\,dx + \int_c^b f(x)\,dx, \qquad \text{where} \quad c = \frac{a+b}{2}$$

We apply Simpson's rule to each piece:

$$Q(a,c) + Q(c,b) \approx \int_a^b f(x)\,dx$$

Error estimation

If we assume the following error

$$Q(a,c) + Q(c,b) - \int_a^b f(x)\,dx = E$$

Then, because Simpson's rule is $\mathcal{O}(h^4)$ accurate, we have that

$$Q(a,b) - \int_a^b f(x)\,dx \approx 16E$$

Then approximately

$$\left(Q(a,b) - \int_a^b f(x)\,dx\right) - \left(Q(a,c) + Q(c,b) - \int_a^b f(x)\,dx\right) \approx 16E - E = 15E$$

Therefore, we get the following error estimate

$$|E| \approx \frac{1}{15}\Big|Q(a,c) + Q(c,b) - Q(a,b)\Big|$$

Basic algorithm

- User provides a, b, $f(x)$, and `TOL`
- The first thing we do is compute the three Simpson's rule approximations:

$$Q(a,b), \quad Q\left(a, \frac{a+b}{2}\right), \quad Q\left(\frac{a+b}{2}, b\right)$$

- We are done if

$$\frac{1}{15}\left|Q(a,c) + Q(c,b) - Q(a,b)\right| < \texttt{TOL}$$

- Otherwise we divide the integral into two pieces:

$$I_{\text{left}} := \int_a^{\frac{a+b}{2}} f(x)\,dx \qquad \text{and} \qquad I_{\text{right}} := \int_{\frac{a+b}{2}}^b f(x)\,dx$$

  For each integral, again compute 3 Simpson's rules and again test the error, this time replacing `TOL` with `0.5*TOL`.

  Why? If I_{left} is accurate to within an error of `0.5*TOL` and I_{right} is accurate to within an error of `0.5*TOL`, then the total error is less than or equal `TOL`.

- Repeat this process recursively . . .

Serial code

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/main.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   double AdaptiveInt(const double a,
                      const double b,
                      const double TOL,
                      char* filename);

   if (argc != 2) { usage(argv[0]); }
   const double TOL = strtod(argv[1], NULL);
   if (TOL<5.0e-16) { usage(argv[0]); }

   const double a = -2.0;
   const double b =  4.0;
   char* filename = "quadrature1.data";
   FILE* file = fopen(filename,"w");
   fclose(file);
```

Serial code

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/main.c:

```
    const double Iex = 0.4147421694070212;
    const double I = AdaptiveInt(a,b,TOL,filename);

    printf("\n");
    printf(" TOL = %24.15e\n",TOL);
    printf("   I = %24.15e\n",I);
    printf(" err = %24.15e\n",fabs(I-Iex));
    printf("\n");

    return 0;
}

void usage(const char *prog_name)
{
    fprintf(stderr,"usage: %s <TOL>\n",prog_name);
    fprintf(stderr,"   TOL should be positive\n");
    exit(1);
}
```

Serial code

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/main.c:

```
double AdaptiveInt(const double a,
                   const double b,
                   const double TOL,
                   char* filename)
{
   double Q(const double a, const double b);

   double Qab = Q(a,b);
   const double c = 0.5*(a+b);
   double Qac = Q(a,c);
   double Qcb = Q(c,b);

   FILE* file = fopen(filename,"a");
   fprintf(file,"%24.15e %24.15e\n",a,b);
   fclose(file);

   const double one_over_15 = 6.666666666666667e-02;
   const double error_est = one_over_15*fabs(Qac+Qcb-Qab);
```

Serial code

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/main.c:

```
    if (error_est<TOL)
    {  return Qac+Qcb;  }
    else
    {
       Qac = AdaptiveInt(a,c,0.5*TOL,filename);
       Qcb = AdaptiveInt(c,b,0.5*TOL,filename);
       return Qac+Qcb;
    }
}

double Q(const double a, const double b)
{
    double f(const double x);
    const double one_sixth = 1.666666666666667e-01;
    const double c = 0.5*(a+b);
    return one_sixth*(b-a)*(f(a)+4.0*f(c)+f(b));
}

double f(const double x)
{
    const double beta = 10.0;
    return exp(-pow(beta*x,2))+sin(x);
}
```

Python plotting routine

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/plotf.py:

```
def plotf():
    # First, figure out how many intervals there are
    fid = open('quadrature1.data', 'r')
    NumIntervals = 0;
    while True:
        line = fid.readline()
        if not line: break
        NumIntervals = NumIntervals+1
    fid.close()

    import numpy as np; import string as str;
    import matplotlib.pyplot as plt

    col = ['b','r','g','m']  # colors for each thread number
```

Python plotting routine

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/plotf.py:

```
    # Second, read-in all the floats
    a = np.zeros(NumIntervals,dtype=float)
    b = np.zeros(NumIntervals,dtype=float)
    fid = open('quadrature1.data', 'r')
    for k in range(0,NumIntervals):
        linestring = fid.readline()
        linelist   = linestring.split()
        a[k]       = float(linelist[0])
        b[k]       = float(linelist[1])
    fid.close()

    # Third, plot the result
    plt.rc("font", size=16);
    plt.figure(1)
    for k in range(0,NumIntervals):
        plt.plot([a[k],b[k]], [-float(k),-float(k)],
            color=col[0], linewidth=2);
    plt.title("Subintervals used in adaptive quadrature")
    plt.axis("off")
    plt.savefig('adapt_quad_ser_1.png', dpi=400, bbox_inches='
        tight')
```

Python plotting routine

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/plotf.py:

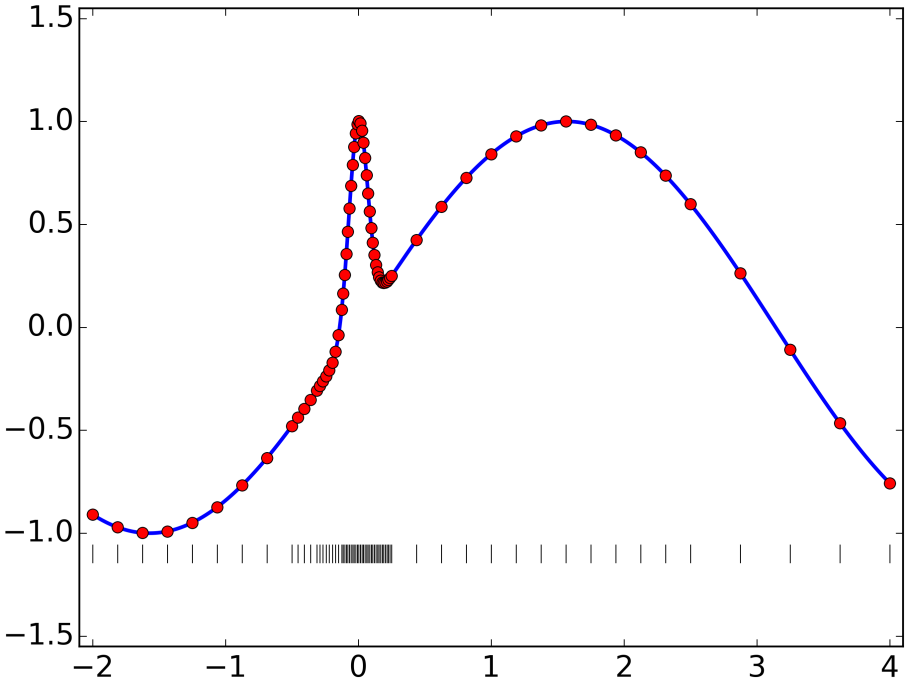
```
# Fourth plot function with intervals
xtmp = np.zeros(5*NumIntervals,dtype=float)
xs = 0
for k in range(0,NumIntervals):
    h = b[k]-a[k]
    xtmp[xs+0] = a[k]
    xtmp[xs+1] = a[k]+0.25*h
    xtmp[xs+2] = a[k]+0.5*h
    xtmp[xs+3] = a[k]+0.75*h
    xtmp[xs+4] = b[k]
    xs = xs+5;
xsorted = np.unique(xtmp);
fsorted = func(xsorted);
xfine = np.linspace(-2.0,4.0,1001);
ffine = func(xfine);
plt.rc("font", size=16);
plt.figure(0)
plt.plot(xfine,ffine,color='b', linewidth=2);
plt.plot(xsorted,fsorted,'o',color='r',linewidth=2);
plt.plot(xsorted,-1.1+0.0*xsorted,'k|',markersize=10);
plt.xlim(-2.1,4.1)
```

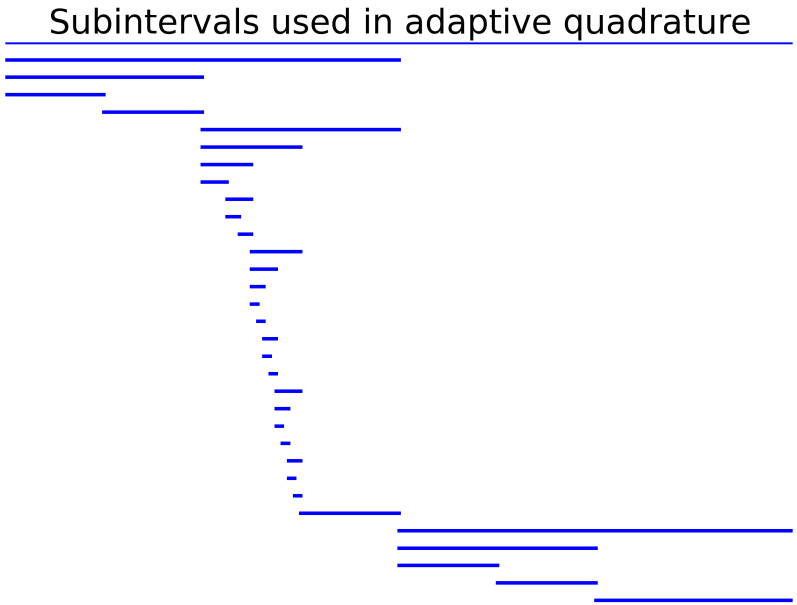
Python plotting routine

$ISUHPC/lectures/lecture20/codes/adaptive_int_serial/plotf.py:

```
    plt.ylim(-1.55,1.55)
    plt.xticks(np.linspace(-2.0,4.0,7))
    plt.yticks(np.linspace(-1.5,1.5,7))
    plt.savefig('adapt_quad_ser_2.png', dpi=400, bbox_inches='
        tight')
    plt.show();

def func(x):

    import numpy as np

    f = np.exp(-(10.0*x)**2)+np.sin(x)

    return f

if __name__ == "__main__":
    plotf();
```

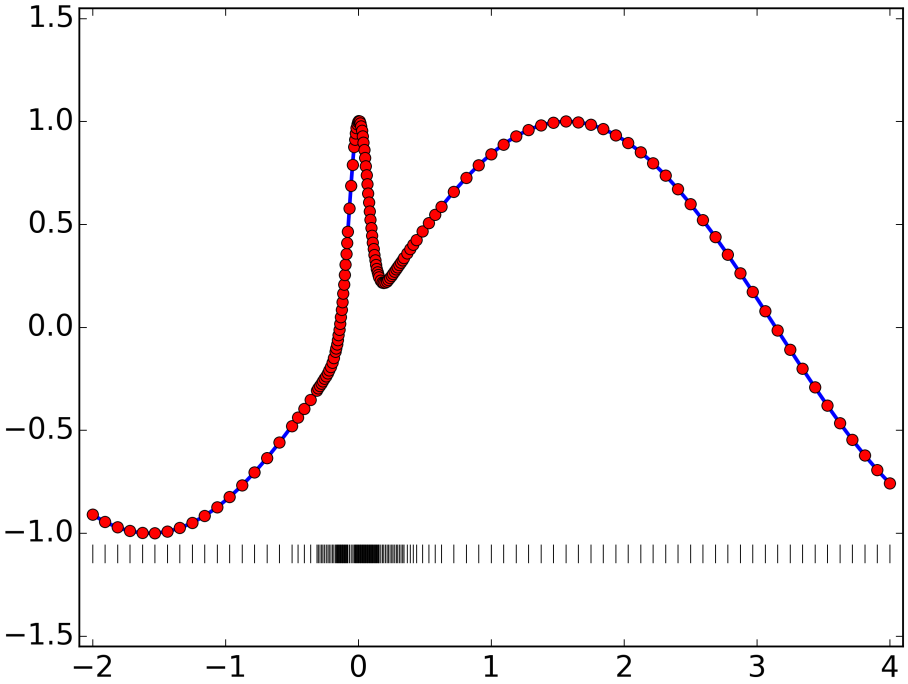
Adaptive quadrature with $\texttt{TOL} = 10^{-4}$

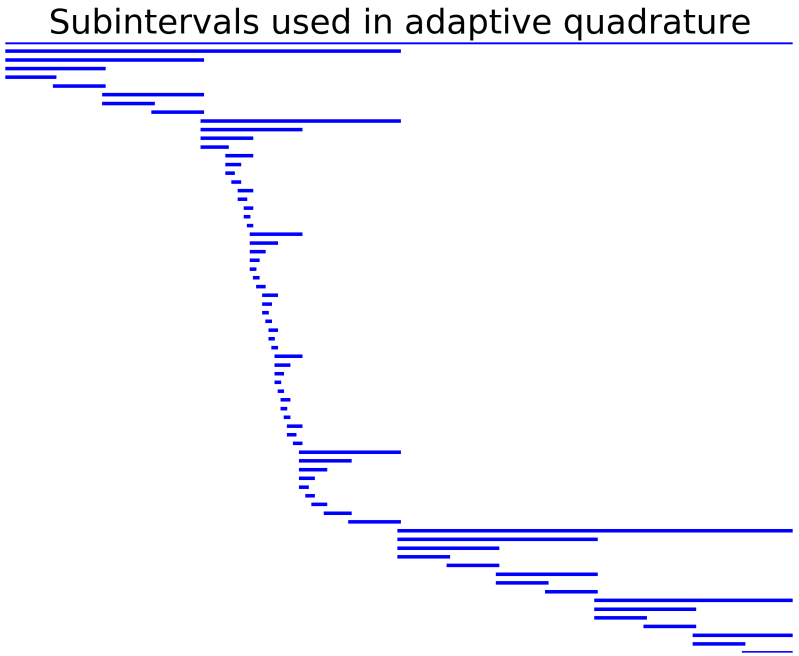





```
$ main.exe 1.0e-4
 TOL =    1.000000000000000e-04
   I =    4.148140671196799e-01
 err =    7.189771265875278e-05

$ python3 plotf.py
```

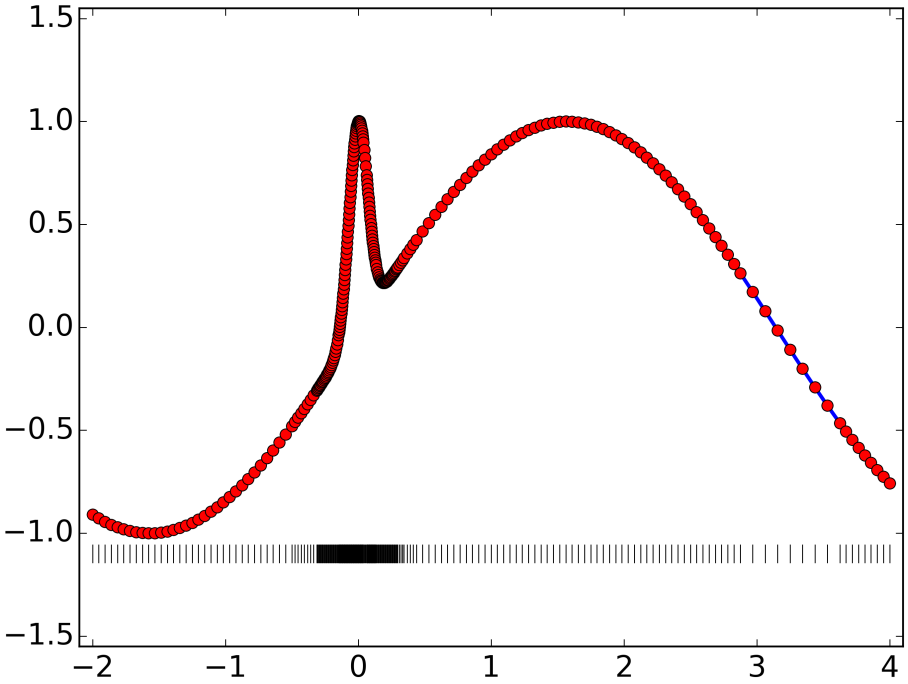
Adaptive quadrature with $\texttt{TOL} = 10^{-5}$

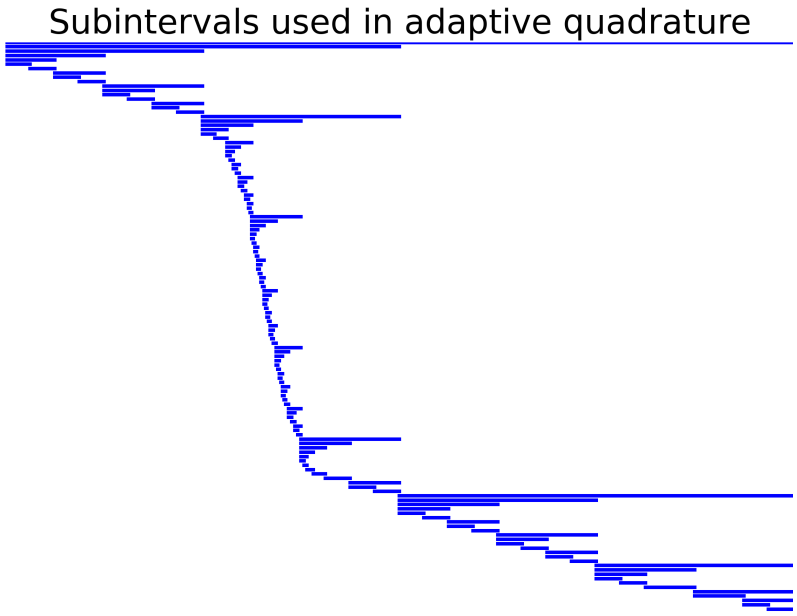





```
$ main.exe 1.0e-5
 TOL =    1.000000000000000e-05
   I =    4.147426081808019e-01
 err =    4.387737806976766e-07

$ python3 plotf.py
```

Adaptive quadrature with $\texttt{TOL} = 10^{-6}$






```
$ main.exe 1.0e-6
 TOL =    1.000000000000000e-06
   I =    4.147421760788315e-01
 err =    6.671810270919565e-09

$ python3 plotf.py
```

OpenMP version #1

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp1/main.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#ifdef _OPENMP
#  include <omp.h>
#endif

int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   double AdaptiveInt(const double a,
                      const double b,
                      const double TOL,
                      char* filename);

   if (argc != 3) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   const double TOL = strtod(argv[2], NULL);
   if (thread_count<0 || TOL<5.0e-16) { usage(argv[0]); }
```

OpenMP version #1

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp1/main.c:

```
    {
       FILE* file = fopen("thread_count.data","w");
       fprintf(file,"%i\n",thread_count);
       fclose(file);
    }

    const double a = -2.0;
    const double b =  4.0;

    for (int i=1; i<=thread_count; i++)
    {
       char filename[36] = "quadrature";
       char file_add[36] = ".data";
       char str[36]; sprintf(str, "%i", i);
       strcat(filename, str);
       strcat(filename, file_add);
       FILE* file = fopen(filename,"w");
       fclose(file);
    }
    const double Iex = 0.4147421694070212;
    double I = 0.0;
```

OpenMP version #1

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp1/main.c:

```
#   pragma omp parallel num_threads(thread_count)
    {
        const int my_rank = omp_get_thread_num();
        char filename[36] = "quadrature";
        char file_add[36] = ".data";
        char str[36]; sprintf(str, "%i", my_rank+1);
        strcat(filename, str);
        strcat(filename, file_add);
        FILE* file = fopen(filename,"a");

        const double width = (b-a)/((double)thread_count);
        const double a_local = a + my_rank*width;
        const double b_local = a_local + width;
        const double TOL_local = TOL/((double)thread_count);

        double I_thread = AdaptiveInt(a_local,b_local,
                                      TOL_local,filename);
        fclose(file);

#       pragma omp critical
        I += I_thread;
    }
```

OpenMP version #1

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp1/main.c:

```
    printf("\n");
    printf(" thread_count = %i\n",thread_count);
    printf(" TOL = %24.15e\n",TOL);
    printf("   I = %24.15e\n",I);
    printf(" err = %24.15e\n",fabs(I-Iex));
    printf("\n");

    return 0;
}

void usage(const char *prog_name)
{
    fprintf(stderr,"usage: %s <num_threads> <TOL>\n",prog_name);
    fprintf(stderr,"   num_threads should be positive\n");
    fprintf(stderr,"   TOL should be positive\n");
    exit(1);
}
```

OpenMP version #1

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp1/main.c:

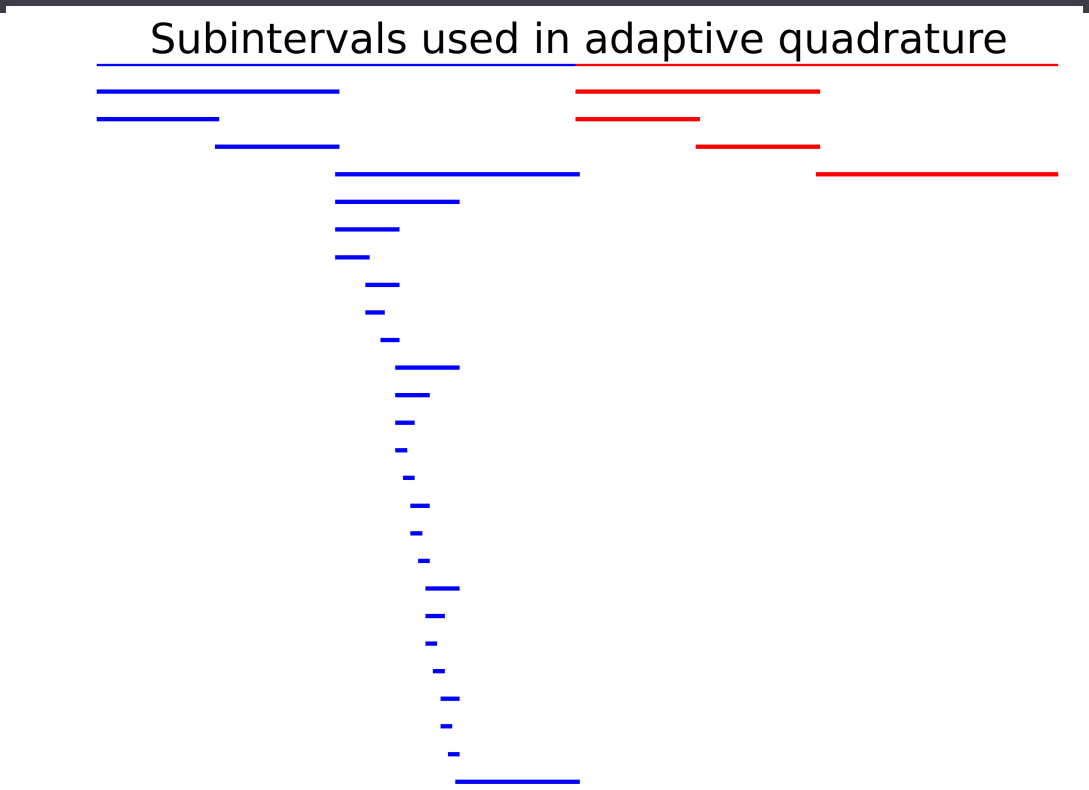
```
double AdaptiveInt(const double a,
                   const double b,
                   const double TOL,
                   char* filename)
{
   double Q(const double a, const double b);

   double Qab = Q(a,b);
   const double c = 0.5*(a+b);
   double Qac = Q(a,c);
   double Qcb = Q(c,b);

   FILE* file = fopen(filename,"a");
   fprintf(file,"%24.15e %24.15e\n",a,b);
   fclose(file);

   const double one_over_15 = 6.666666666666667e-02;
   const double error_est = one_over_15*fabs(Qac+Qcb-Qab);
```

OpenMP version #1

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp1/main.c:

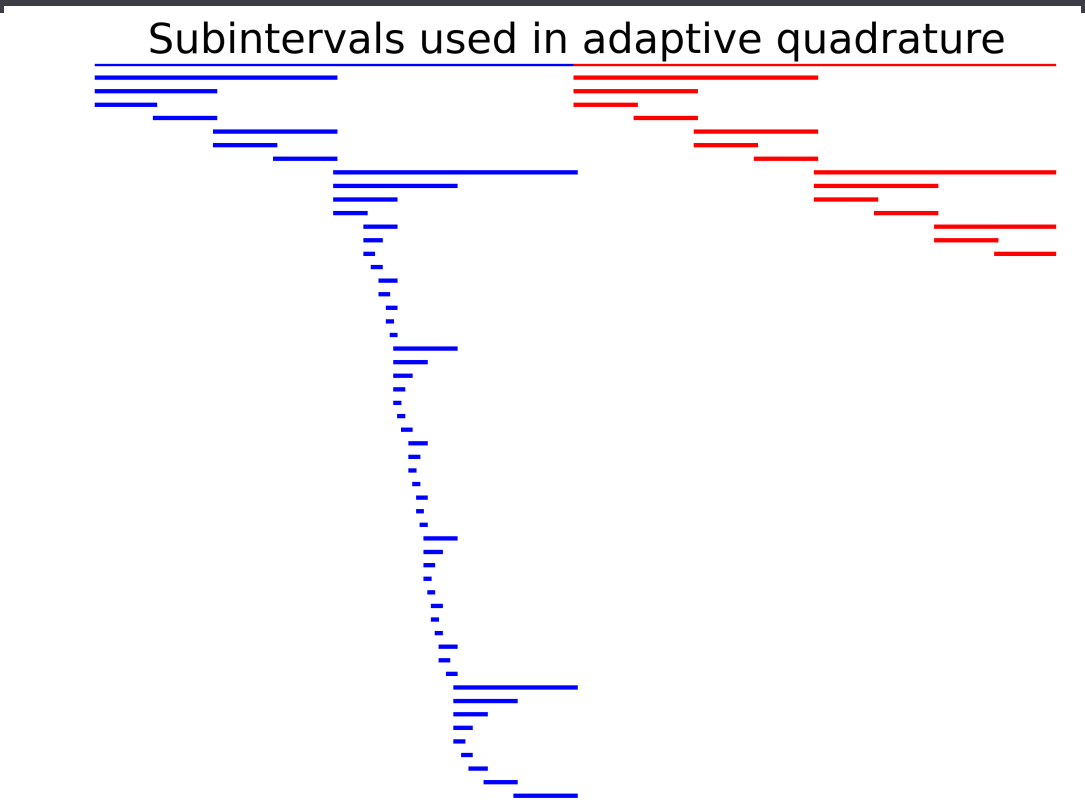
```
    if (error_est<TOL)
    {  return Qac+Qcb;  }
    else
    {
       Qac = AdaptiveInt(a,c,0.5*TOL,filename);
       Qcb = AdaptiveInt(c,b,0.5*TOL,filename);
       return Qac+Qcb;
    }
}

double Q(const double a, const double b)
{
    double f(const double x);
    const double one_sixth = 1.666666666666667e-01;
    const double c = 0.5*(a+b);
    return one_sixth*(b-a)*(f(a)+4.0*f(c)+f(b));
}

double f(const double x)
{
    const double beta = 10.0;
    return exp(-pow(beta*x,2))+sin(x);
}
```

Adaptive quadrature with $\texttt{TOL} = 10^{-4}$



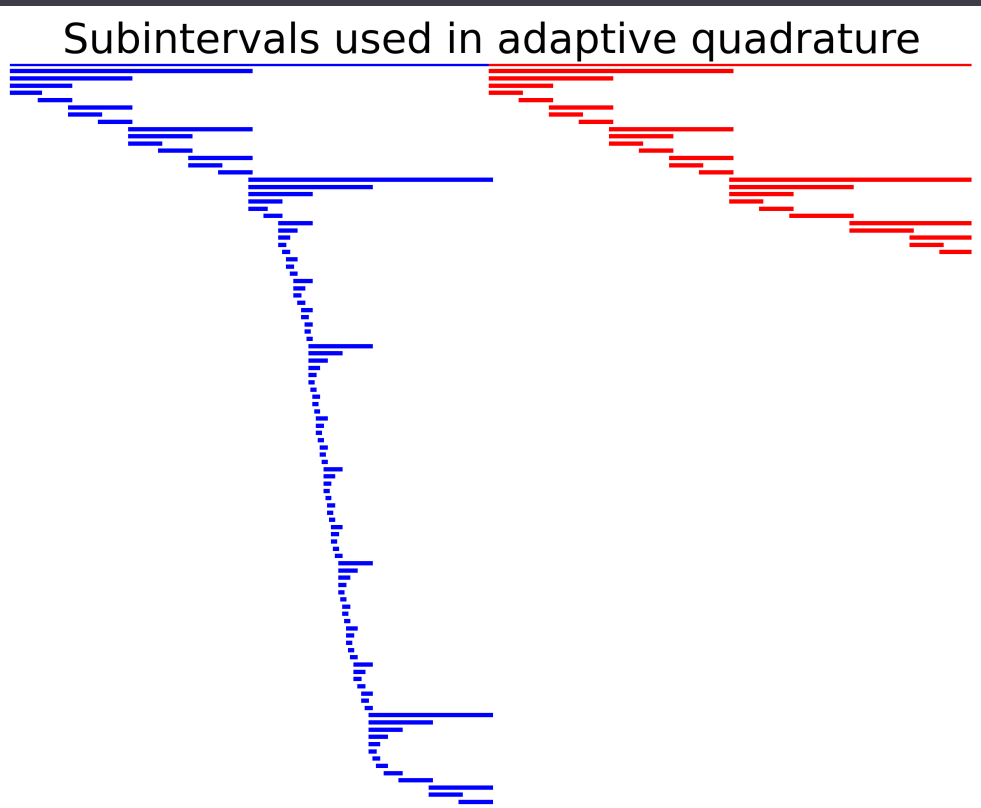

```
$ main.exe 2 1.0e-4
 thread_count = 2
 TOL =    1.000000000000000e-04
   I =    4.148140671196799e-01
 err =    7.189771265875278e-05

$ python3 plotf.py
```

Adaptive quadrature with $\texttt{TOL} = 10^{-5}$




```
$ main.exe 2 1.0e-5
 thread_count = 2
 TOL =    1.000000000000000e-05
   I =    4.147426081808019e-01
 err =    4.387737806976766e-07

$ python3 plotf.py
```

Adaptive quadrature with $\texttt{TOL} = 10^{-6}$




```
$ main.exe 2 1.0e-6
 thread_count = 2
 TOL =    1.000000000000000e-06
   I =    4.147421760788315e-01
 err =    6.671810270919565e-09

$ python3 plotf.py
```

OpenMP version #2 (Nested Parallelization)

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp2/main.c:

```
int main(int argc, char* argv[])
{
   void usage(const char* prog_name);
   double AdaptiveInt(const double a,
                      const double b,
                      const double TOL,
                      char* filename,
                      const int num_threads);
   double AdaptiveIntSerial(const double a,
                            const double b,
                            const double TOL,
                            char* filename);

   if (argc != 3) { usage(argv[0]); }
   const int thread_count = strtol(argv[1], NULL, 10);
   const double TOL = strtod(argv[2], NULL);
   if (thread_count<0 || TOL<5.0e-16) { usage(argv[0]); }

   const double a = -2.0;
   const double b =  4.0;

   char filename[36] = "quadrature.data";
   FILE* file = fopen(filename,"w");
```

OpenMP version #2 (Nested Parallelization)

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp2/main.c:

```
    FILE* file = fopen(filename,"w");
    fclose(file);

    const double Iex = 0.4147421694070212;

    // allow for nested OpenMP
    //omp_set_nested(1); omp_set_dynamic(0); //deprecated
    omp_set_max_active_levels(omp_get_supported_active_levels()
        ); omp_set_dynamic(0);

    // call adaptive integration recursively
    double I;
    if (thread_count==1)
    {  I = AdaptiveIntSerial(a,b,TOL,filename);  }
    else
    {  I = AdaptiveInt(a,b,TOL,filename,thread_count);  }

    // print some data to the screen
    printf("\n thread_count = %i\n",thread_count);
    printf(" TOL = %24.15e\n",TOL);
    printf("   I = %24.15e\n",I);
    printf(" err = %24.15e\n\n",fabs(I-Iex));

    return 0;
```

OpenMP version #2 (Nested Parallelization)

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp2/main.c:

```c

void usage(const char *prog_name)
{
   fprintf(stderr,"usage: %s <num_threads> <TOL>\n",prog_name);
   fprintf(stderr,"   num_threads should be positive\n");
   fprintf(stderr,"   TOL should be positive\n");
   exit(1);
}

double AdaptiveIntSerial(const double a,
                         const double b,
                         const double TOL,
                         char* filename)
{
   double Q(const double a, const double b);
   double Qab = Q(a,b);
   const double c = 0.5*(a+b);
   double Qac = Q(a,c);
   double Qcb = Q(c,b);
   const int my_rank = omp_get_thread_num();
   FILE* file = fopen(filename,"a");
   fprintf(file,"%3i %24.15e %24.15e\n",my_rank+1,a,b);
```

OpenMP version #2 (Nested Parallelization)

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp2/main.c:

```
    fclose(file);
    const double one_over_15 = 6.666666666666667e-02;
    const double error_est = one_over_15*fabs(Qac+Qcb-Qab);
    double result = 0.0;
    if (error_est<TOL)
    { result = Qac+Qcb; }
    else
    {
       result =  AdaptiveIntSerial(a,c,0.5*TOL,filename);
       result += AdaptiveIntSerial(c,b,0.5*TOL,filename);
    }
    return result;
 }

 double AdaptiveInt(const double a,
                    const double b,
                    const double TOL,
                    char* filename,
                    const int num_threads)
 {
    double Q(const double a, const double b);
    double Qab = Q(a,b);
```

OpenMP version #2 (Nested Parallelization)

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp2/main.c:

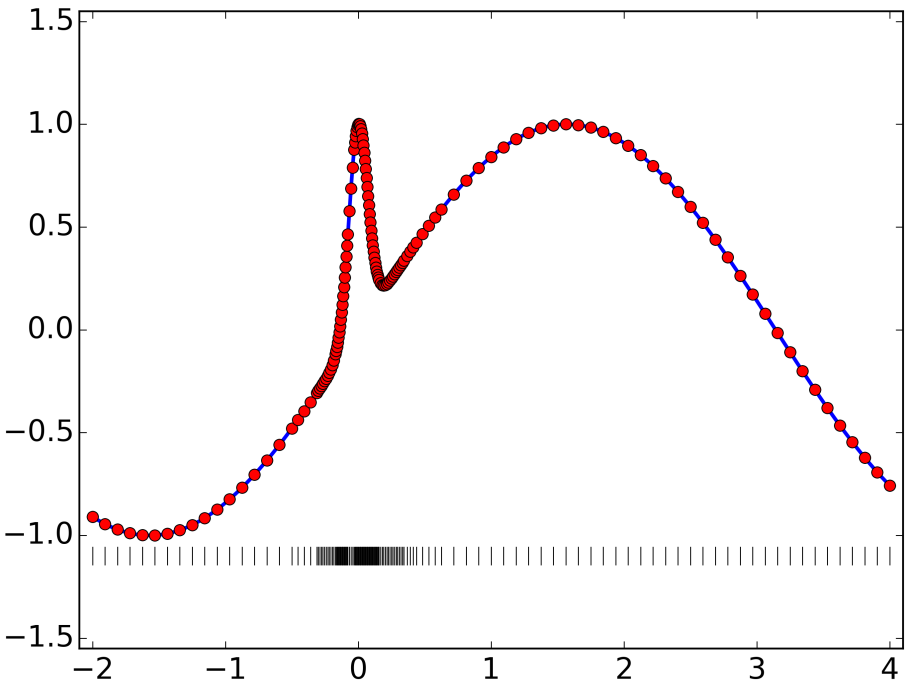
```
    const double c = 0.5*(a+b);
    double Qac = Q(a,c);
    double Qcb = Q(c,b);
    const int my_rank = omp_get_thread_num();
    FILE* file = fopen(filename,"a");
    fprintf(file,"%3i %24.15e %24.15e\n",my_rank+1,a,b);
    fclose(file);
    const double one_over_15 = 6.666666666666667e-02;
    const double error_est = one_over_15*fabs(Qac+Qcb-Qab);
    double result = 0.0;
    const double one_over = 1.0/((double)num_threads);
    if (error_est<TOL)
    { result = Qac+Qcb; }
    else
    {
       result = 0.0;
#      pragma omp parallel num_threads(num_threads)
       {
          const int my_rank_local = omp_get_thread_num();
          double a_local = a + my_rank_local*(b-a)*one_over;
          double b_local = a_local + (b-a)*one_over;
          double Q = AdaptiveInt(a_local,b_local,
                                 one_over*TOL,
```

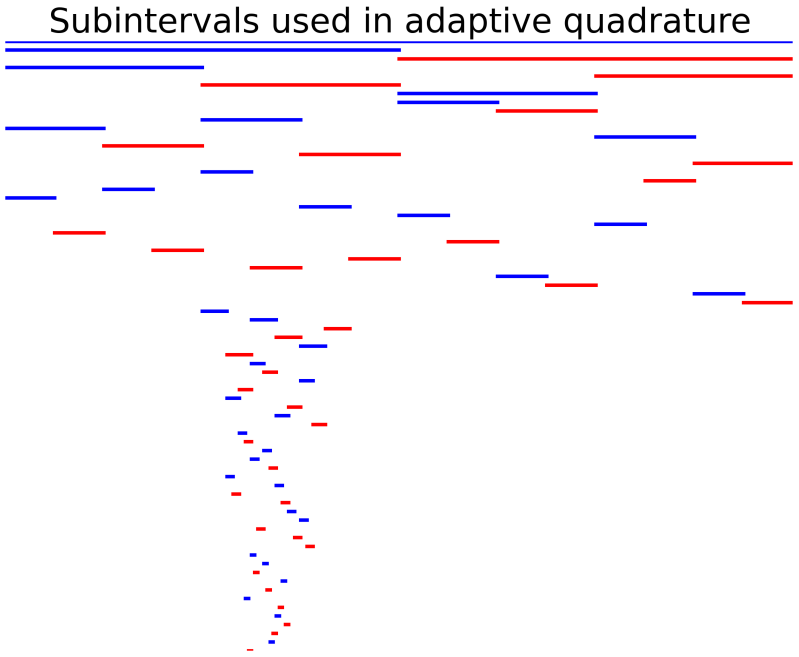
OpenMP version #2 (Nested Parallelization)

$ISUHPC/lectures/lecture20/codes/adaptive_int_omp2/main.c:

```
                                        filename,num_threads);
#           pragma omp critical
            result += Q;
        }
    }
    return result;
}

double Q(const double a, const double b)
{
    double f(const double x);
    const double one_sixth = 1.666666666666667e-01;
    const double c = 0.5*(a+b);
    return one_sixth*(b-a)*(f(a)+4.0*f(c)+f(b));
}

double f(const double x)
{
    const double beta = 10.0;
    return exp(-pow(beta*x,2))+sin(x);
}
```

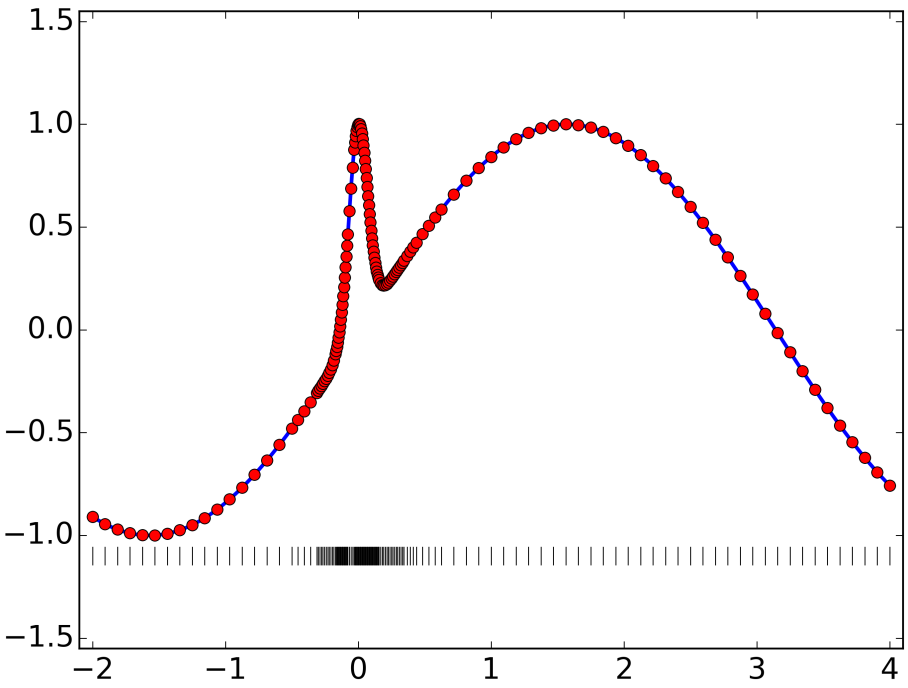
Adaptive quadrature with TOL $= 10^{-5}$

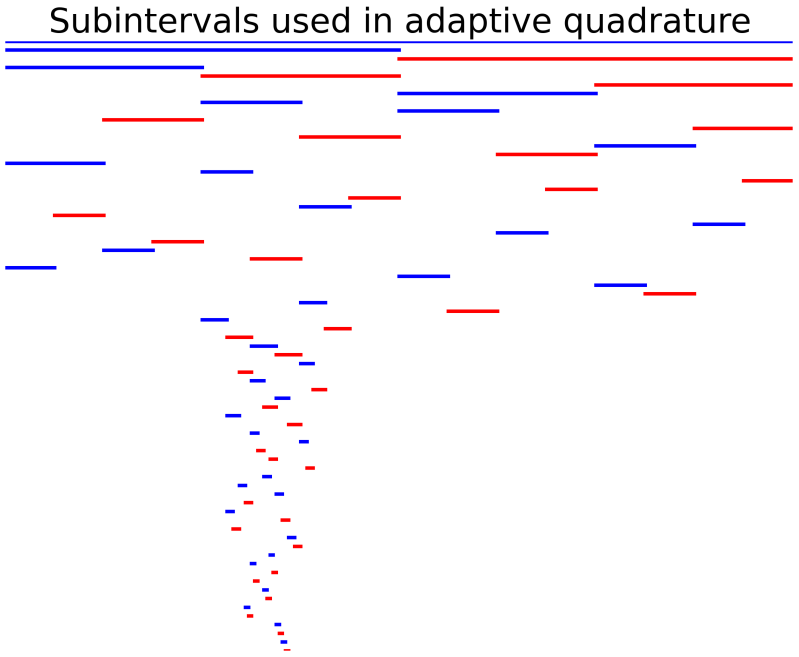





```
$ main.exe 2 1.0e-5
 thread_count = 2
 TOL =    1.000000000000000e-05
   I =    4.147426081808019e-01
 err =    4.387737806976766e-07

$ python3 plotf.py
```

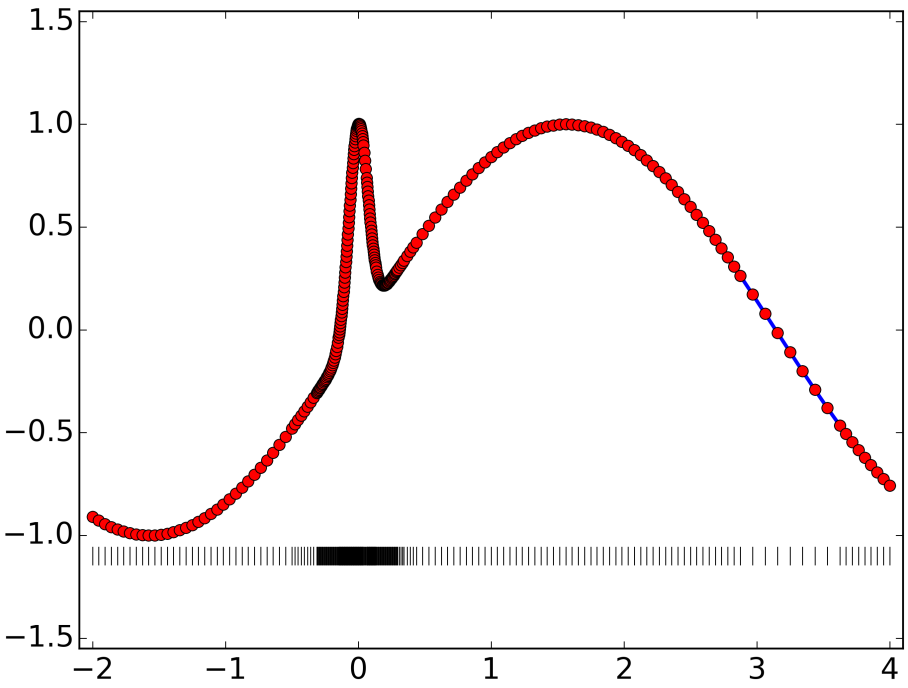
Adaptive quadrature with $\texttt{TOL} = 10^{-5}$

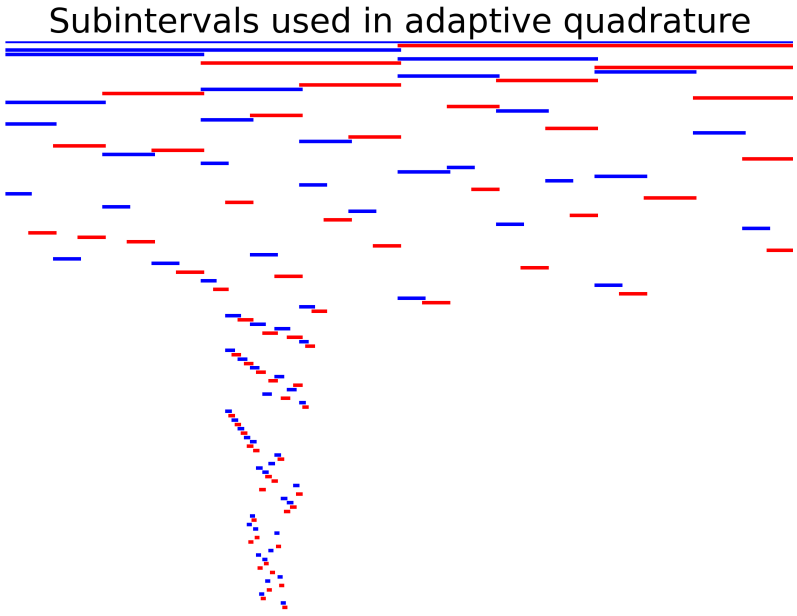





```
$ main.exe 2 1.0e-5
 thread_count = 2
 TOL =    1.000000000000000e-05
   I =    4.147426081808019e-01
 err =    4.387737806976766e-07

$ python3 plotf.py
```

Adaptive quadrature with TOL $= 10^{-6}$






```
$ main.exe 2 1.0e-6
 thread_count = 2
 TOL =    1.000000000000000e-06
   I =    4.147421760788315e-01
 err =    6.671810270919565e-09

$ python3 plotf.py
```

Lab Assignment

- Computing 1D Electrostatic Potential

$$V(x_0) = \int_a^b \frac{\rho(x)}{|x - x_0|} dx = \int_a^b \frac{\rho(x) - \rho(x_0)}{|x - x_0|} dx + \rho(x_0) \int_a^b \frac{1}{|x - x_0|} dx$$
$$= \int_a^b \frac{\rho(x) - \rho(x_0)}{|x - x_0|} dx + \rho(x_0) \log \frac{b - x_0}{x_0 - a}$$

  for $x_0 \in (a, b)$. Choose $\rho(x) = e^{-x^2}$ over $[-1, 1]$. Plot V.

- Compute the Bessel function,

$$J_0(x) = \frac{1}{\pi} \int_0^\pi \cos(x \sin \theta) d\theta,$$

  for $x \in [0, 50]$, Plot J_0.

- Submit source code, screenshots, and update Git.